

```

/*
 * ITECC04 - Data Structures and Algorithms
 * PART II - Music Playlist Manager
 * Data Structure: Singly Linked List ADT
 * (self-implemented, no built-in
 * ArrayList/LinkedList/Vector/Stack/Queue
 * is used)
 */

import java.util.Scanner;

// -----
// -----
// Song: holds the information of a single
// song
// -----
// -----
class Song {
    String songId;
    String songTitle;
    String artist;
    String duration; // format: mm:ss

    public Song(String songId, String
songTitle, String artist, String duration)
    {
        this.songId = songId;
        this.songTitle = songTitle;
        this.artist = artist;
        this.duration = duration;
    }
}

```

```

    }

    @Override
    public String toString() {
        return "Song ID: " + songId
            + " | Title: " + songTitle
            + " | Artist: " + artist
            + " | Duration: " +
duration;
    }
}

// -----
// Node: a single link in the chain, holds
// Song data + next reference
// -----
class Node {
    Song data;
    Node next;

    public Node(Song data) {
        this.data = data;
        this.next = null;
    }
}

// -----
// LinkedList: the self-implemented Singly
// Linked List ADT
// that manages the Node objects (manual
// node creation/linking)

```

```
// -----  
-----  
class LinkedList {  
    private Node head;  
    private Node tail;  
    private int count;  
  
    public LinkedList() {  
        head = null;  
        tail = null;  
        count = 0;  
    }  
  
    public boolean isEmpty() {  
        return head == null;  
    }  
  
    public int size() {  
        return count;  
    }  
  
    // 1. Add Song at the Beginning  
    public void insertFirst(Song song) {  
        Node newNode = new Node(song);  
        if (isEmpty()) {  
            head = newNode;  
            tail = newNode;  
        } else {  
            newNode.next = head;  
            head = newNode;  
        }  
        count++;  
    }  
}
```

```

// 2. Add Song at the End
public void insertLast(Song song) {
    Node newNode = new Node(song);
    if (isEmpty()) {
        head = newNode;
        tail = newNode;
    } else {
        tail.next = newNode;
        tail = newNode;
    }
    count++;
}

// 3. Insert Song at a Specific
Position (1-based position)
public boolean insertAt(int position,
Song song) {
    if (position < 1 || position >
count + 1) {
        return false; // invalid
position
    }
    if (position == 1) {
        insertFirst(song);
        return true;
    }
    if (position == count + 1) {
        insertLast(song);
        return true;
    }

    Node newNode = new Node(song);
    Node current = head;
    for (int i = 1; i < position - 1;

```

```

i++) {
    current = current.next;
}
newNode.next = current.next;
current.next = newNode;
count++;
return true;
}

// 6. Remove Song (by Song ID),
properly relinking the nodes
public boolean delete(String songId) {
    if (isEmpty()) {
        return false;
    }

    if
(head.data.songId.equalsIgnoreCase(songId))
{
        head = head.next;
        if (head == null) {
            tail = null;
        }
        count--;
        return true;
    }

    Node current = head;
    while (current.next != null) {
        if
(current.next.data.songId.equalsIgnoreCase(
songId)) {
            current.next =
current.next.next;

```

```

        if (current.next == null) {
            tail = current;
        }
        count--;
        return true;
    }
    current = current.next;
}
return false; // not found
}

// 5. Search Song (by Song ID)
public Song search(String songId) {
    Node current = head;
    while (current != null) {
        if
(current.data.songId.equalsIgnoreCase(songI
d)) {
            return current.data;
        }
        current = current.next;
    }
    return null; // not found
}

// 4. Display Playlist (traverse from
head until NULL)
public void display() {
    if (isEmpty()) {
        System.out.println("The
playlist is currently empty.");
        return;
    }
    System.out.println("-----

```

```

-----
-----");
        int num = 1;
        Node current = head;
        while (current != null) {
            System.out.println(num + ". " +
current.data);
            current = current.next;
            num++;
        }
        System.out.println("-----
-----");
        System.out.println("Total number of
songs: " + count);
    }
}

// -----
-----
// Main: console menu that drives the Music
Playlist Manager
// -----
-----
public class Main {
    static LinkedList playlist = new
LinkedList();
    static Scanner sc = new
Scanner(System.in);

    public static void main(String[] args)
{
        loadSampleSongs();

```

```

        int choice;
        do {
            printMenu();
            choice = getIntInput("Enter
your choice: ");

            switch (choice) {
                case 1:
addSongAtBeginning(); break;
                case 2: addSongAtEnd();
break;
                case 3:
insertSongAtPosition(); break;
                case 4: displayPlaylist();
break;
                case 5: searchSong();
break;
                case 6: removeSong();
break;
                case 7:
displayPlaylistSize(); break;
                case 8:
System.out.println("Exiting Music Playlist
Manager. Goodbye!"); break;
                default:
System.out.println("Invalid choice. Please
try again.");
            }
            System.out.println();
        } while (choice != 8);

        sc.close();
    }

```

```

        // Pre-loaded songs so the playlist
        isn't empty on first run
        static void loadSampleSongs() {
            playlist.insertLast(new
            Song("S001", "Perfect", "Ed Sheeran",
            "4:23"));
            playlist.insertLast(new
            Song("S002", "Blinding Lights", "The
            Weeknd", "3:20"));
            playlist.insertLast(new
            Song("S003", "Let Her Go", "Passenger",
            "4:12"));
            playlist.insertLast(new
            Song("S004", "Pasilyo", "SUD", "4:01"));
            playlist.insertLast(new
            Song("S005", "Someone Like You", "Adele",
            "4:45"));
            playlist.insertLast(new
            Song("S006", "Titibo-Tibo", "Moirira Dela
            Torre", "4:08"));
        }

        static void printMenu() {

            System.out.println("=====
            =====");
            System.out.println("        MUSIC
            PLAYLIST MANAGER");

            System.out.println("=====
            =====");
            System.out.println("1. Add Song at
            Beginning");
            System.out.println("2. Add Song at

```

```

End");
        System.out.println("3. Insert Song
at Position");
        System.out.println("4. Display
Playlist");
        System.out.println("5. Search
Song");
        System.out.println("6. Remove
Song");
        System.out.println("7. Display
Playlist Size");
        System.out.println("8. Exit");
    }

    static int getIntInput(String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            System.out.print("Invalid
input. Please enter a number: ");
            sc.next();
        }
        int value = sc.nextInt();
        sc.nextLine(); // consume leftover
newline
        return value;
    }

    static Song inputSongDetails() {
        System.out.print("Enter Song ID:
");
        String id = sc.nextLine();
        System.out.print("Enter Song Title:
");
        String title = sc.nextLine();
    }

```

```

        System.out.print("Enter Artist: ");
        String artist = sc.nextLine();
        System.out.print("Enter Duration
(mm:ss): ");
        String duration = sc.nextLine();
        return new Song(id, title, artist,
duration);
    }

    static void addSongAtBeginning() {
        Song s = inputSongDetails();
        playlist.insertFirst(s);
        System.out.println("Song added at
the beginning of the playlist.");
    }

    static void addSongAtEnd() {
        Song s = inputSongDetails();
        playlist.insertLast(s);
        System.out.println("Song added at
the end of the playlist.");
    }

    static void insertSongAtPosition() {
        int pos = getIntInput("Enter
position (1 to " + (playlist.size() + 1) +
"): ");
        Song s = inputSongDetails();
        boolean success =
playlist.insertAt(pos, s);
        if (success) {
            System.out.println("Song
inserted at position " + pos + ".");
        } else {

```

```
        System.out.println("Invalid
position. Insertion failed.");
    }
}

static void displayPlaylist() {
    playlist.display();
}

static void searchSong() {
    System.out.print("Enter Song ID to
search: ");
    String id = sc.nextLine();
    Song found = playlist.search(id);
    if (found != null) {
        System.out.println("Song
found!");
        System.out.println(found);
    } else {
        System.out.println("No song
found with ID " + id + ".");
    }
}

static void removeSong() {
    System.out.print("Enter Song ID to
remove: ");
    String id = sc.nextLine();
    boolean removed =
playlist.delete(id);
    if (removed) {
        System.out.println("Song
removed successfully.");
    } else {
```

```
        System.out.println("No song  
found with ID " + id + ".");  
    }  
}  
  
    static void displayPlaylistSize() {  
        System.out.println("Total songs  
currently in the playlist: " +  
playlist.size());  
    }  
}
```